

Pipeline de Dados Automatizado de Vendas (ETL) - Diego Teodoro

Este documento consolida o projeto de Pipeline ETL desenvolvido por Diego Teodoro, apresentando a arquitetura, o código-fonte e a documentação técnica, tudo em português para facilitar a apresentação.



Visão Geral do Projeto

Este repositório apresenta um **Pipeline ETL Automatizado** de nível de produção, projetado para processar grandes volumes de dados de vendas e clientes. O projeto implementa uma arquitetura robusta que abrange desde a extração e limpeza de dados até transformações de negócios complexas e orquestração automatizada.

O objetivo principal é transformar dados brutos e fragmentados em conjuntos de dados analíticos estruturados e de alta qualidade, permitindo a tomada de decisões baseada em dados por meio de um fluxo de trabalho automatizado e monitorado.



Arquitetura e Design

O projeto segue uma arquitetura modular e desacoplada, garantindo que cada componente do processo ETL seja independente, testável e escalável.

Componente	Responsabilidade	Tecnologia
Extração	Ingestão de dados de APIs REST, arquivos CSV, JSON e Parquet.	Requests , Pandas
Transformação	Limpeza de dados, padronização, aplicação de lógica de negócios e enriquecimento.	Pandas , NumPy
Validação	Garantia da integridade e qualidade dos dados por meio de verificações automatizadas.	Motor de Validação Customizado
Carga	Persistência de dados processados em bancos de dados relacionais e formatos de arquivo otimizados.	SQLAlchemy , PyArrow , DuckDB
Orquestração	Agendamento e monitoramento do fluxo de trabalho de ponta a ponta.	Apache Airflow

Principais Recursos de Engenharia

- **Extração Resiliente:** Lógica de backoff exponencial e retentativa implementada para chamadas de API usando `tenacity`.
- **Framework de Qualidade de Dados:** Camada de validação integrada que verifica a consistência do esquema, limites de nulos e intervalos de valores antes do carregamento.
- **Logging Estruturado:** Logs formatados em JSON para melhor observabilidade e integração com pilhas de monitoramento (ELK/Datadog).
- **Armazenamento Eficiente:** Utiliza Parquet para snapshots analíticos, reduzindo o espaço de armazenamento e melhorando o desempenho da consulta. Integração com **DuckDB** para análises OLAP rápidas.
- **Operações Idempotentes:** Projetado para ser executado novamente com segurança, sem duplicar dados ou causar inconsistências.
- **Testes Automatizados:** Cobertura de testes unitários para garantir a funcionalidade e robustez do código.

Estrutura do Projeto

```
.
├── airflow/                # Camada de Orquestração
│   └── dags/               # Definições de DAGs do Airflow
├── data/                   # Simulação de Data Lake
│   ├── input/             # Zona de Aterrissagem (Dados Brutos)
│   └── output/            # Zona Curada (Dados Processados)
├── src/                    # Lógica Central do ETL
│   ├── config/            # Configuração de Ambiente e Logging
│   ├── db_connection/     # Conectores de Banco de Dados
│   ├── extract/           # Módulos de Ingestão
│   ├── transform/        # Lógica de Processamento e Negócios
│   ├── quality/          # Motor de Qualidade de Dados
│   ├── load/             # Módulos de Persistência
│   └── main.py            # Ponto de Entrada do Pipeline
├── tests/                 # Suíte de Testes Automatizados
│   ├── test_extract_data.py
│   ├── test_transform_data.py
│   ├── test_data_validator.py
│   └── test_sqlite_connector.py
├── .env.example           # Modelo de Configuração
├── requirements-core.txt   # Dependências principais do projeto
└── requirements-airflow.txt # Dependências específicas do Airflow
```

Primeiros Passos

Pré-requisitos

- Python 3.11+
- [Apache Airflow](#) (para orquestração)

Instalação

1. Clone o repositório:

```
git clone https://github.com/TeOdor0/automated-etl-pipeline.git
cd automated-etl-pipeline
```

2. Configure um ambiente virtual:

```
python -m venv venv
source venv/bin/activate # No Windows: venv\Scripts\activate
```

3. Instale as dependências:

```
pip install -r requirements-core.txt
# Para Airflow, instale as dependências separadamente:
# pip install -r requirements-airflow.txt
```

4. Configure as variáveis de ambiente:

```
cp .env.example .env
# Edite .env com suas configurações específicas
```

Executando o Pipeline

Execução Manual:

```
python src/main.py
```

Execução Automatizada (Airflow): Copie o conteúdo de `airflow/dags/` para a pasta de DAGs do seu Airflow e acione a DAG `etl_sales_pipeline` através da UI.



Monitoramento e Qualidade

O pipeline gera relatórios de execução e logs detalhados. Em caso de falhas, o sistema oferece:

- Contexto de erro detalhado em `logs/error.log`.
- Relatórios de validação de dados destacando problemas específicos de integridade.
- Retentativas automáticas para falhas transitórias.
- Métricas de execução salvas em `data/output/pipeline_metrics.json`.



Contato

Diego Teodoro

- [LinkedIn](#)
- [GitHub](#)

Este projeto foi desenvolvido para demonstrar habilidades avançadas em Engenharia de Dados usando o ecossistema Python.



Código-Fonte do Projeto

Abaixo estão os scripts principais que compõem o pipeline, organizados por módulo.

src/main.py - Orquestrador Principal

```
"""
Pipeline de Dados ETL Automatizado
Desenvolvido por: Diego Teodoro

Este script orquestra o fluxo completo de extração, transformação,
validação, carga e geração de métricas de dados de vendas e clientes.
Ele foi projetado para ser robusto, observável e pronto para produção.
"""

import sys
from pathlib import Path
import json
from datetime import datetime

# Adicionar o diretório raiz ao path
sys.path.insert(0, str(Path(__file__).parent.parent))

from src.extract.extract_data import DataExtractor
from src.transform.transform_data import DataTransformer
from src.quality.data_validator import DataValidator
from src.load.load_data import DataLoader
from src.config import ETLLogger, OUTPUT_DIR

def run_etl_pipeline():
    """
    Executa o pipeline completo de vendas e clientes, orquestrando as etapas
    de extração, transformação, validação, carga e geração de métricas.
    """
    logger = ETLLogger(__name__)
    logger.logger.info("=" * 60)
    logger.logger.info("INICIANDO PROCESSAMENTO ETL - BY DIEGO TEODORO")
    logger.logger.info("=" * 60)

    try:
        # =====
        # ETAPA 1: EXTRACT (Extração)
        # =====
        logger.logger.info("\n[ETAPA 1] EXTRAÇÃO DE DADOS (Extract)")
        logger.logger.info("-" * 60)

        extractor = DataExtractor()

        # Extração de dados de vendas (Mock Data para demonstração)
```

```

df_sales = extractor.extract_mock_sales_data(num_records=100)
logger.logger.info(f"✓ Vendas extraídas: {len(df_sales)}
registros")

# Extração de dados de clientes (Mock Data para demonstração)
df_customers = extractor.extract_mock_customers_data(num_records=50)
logger.logger.info(f"✓ Clientes extraídos: {len(df_customers)}
registros")

# =====
# ETAPA 2: TRANSFORM (Transformação)
# =====
logger.logger.info("\n[ETAPA 2] TRANSFORMAÇÃO DE DADOS (Transform)")
logger.logger.info("-" * 60)

transformer = DataTransformer()

# 2.1 Tratar valores ausentes antes de outras transformações
df_sales = transformer.handle_missing_values(
    df_sales,
    strategy='fill',
    columns=["quantity", "unit_price", "discount"],
    fill_value=0
)
logger.logger.info("✓ Valores ausentes em vendas tratados")

# 2.2 Padronizar nomes de colunas
df_sales = transformer.standardize_column_names(df_sales)
df_customers = transformer.standardize_column_names(df_customers)
logger.logger.info("✓ Nomes de colunas padronizados")

# 2.3 Garantir tipos de dados corretos
df_sales = transformer.enforce_data_types(
    df_sales,
    {
        'sale_id': 'int',
        'customer_id': 'int',
        'product_id': 'int',
        'sale_date': 'datetime',
        'quantity': 'int',
        'unit_price': 'float',
        'discount': 'float'
    }
)
logger.logger.info("✓ Tipos de dados das vendas corrigidos")

```

```

df_customers = transformer.enforce_data_types(
    df_customers,
    {
        \'customer_id\': \'int\',
        \'registration_date\': \'datetime\'
    }
)
logger.logger.info("✓ Tipos de dados dos clientes corrigidos")

# 2.4 Adicionar colunas calculadas
df_sales = transformer.add_calculated_columns(
    df_sales,
    {
        \'subtotal\': \'quantity * unit_price\',
        \'discount_amount\': \'quantity * unit_price * discount\',
        \'total_amount\': \'quantity * unit_price * (1 - discount)\'
    }
)
logger.logger.info("✓ Colunas calculadas adicionadas")

# 2.5 Filtrar apenas vendas completadas
df_sales_completed = transformer.filter_data(
    df_sales,
    "status == \'completed\'"
)
logger.logger.info(f"✓ Filtradas {len(df_sales_completed)} vendas
completadas")

# 2.6 Remover duplicatas
df_sales_completed = transformer.remove_duplicates(
    df_sales_completed,
    subset=[\'sale_id\']
)
logger.logger.info("✓ Duplicatas removidas")

# 2.7 Fazer join entre vendas e clientes
df_enriched = transformer.join_dataframes(
    df_sales_completed,
    df_customers,
    on=\'customer_id\',
    how=\'left\'
)
logger.logger.info(f"✓ Join realizado: {len(df_enriched)}
registros")

# 2.8 Criar agregação por cliente

```



```

df_customer_summary = transformer.aggregate_data(
    df_enriched,
    group_by=[\'customer_id\', \'first_name\', \'last_name\',
\'city\'],
    aggregations={
        \'total_amount\': \'sum\',
        \'sale_id\': \'count\',
        \'quantity\': \'sum\'
    }
)
df_customer_summary.columns = [
    \'customer_id\', \'first_name\', \'last_name\', \'city\',
    \'total_revenue\', \'num_sales\', \'total_items\'
]
logger.logger.info(f"✓ Agregação por cliente:
{len(df_customer_summary)} clientes")

# =====
# ETAPA 3: QUALITY (Validação de Qualidade)
# =====
logger.logger.info("\n[ETAPA 3] VALIDAÇÃO DE QUALIDADE (Quality)")
logger.logger.info("-" * 60)

validator = DataValidator()

# Validar dados enriquecidos
validation_config = {
    "required_columns": [\'sale_id\', \'customer_id\',
\'total_amount\'],
    "expected_types": {
        \'sale_id\': int,
        \'customer_id\': int,
        \'total_amount\': float
    },
    "value_ranges": {
        \'total_amount\': {\'min\': 0},
        \'quantity\': {\'min\': 1}
    }
}

validation_summary = validator.run_all_validations(
    df_enriched,
    validation_config
)

logger.logger.info(f"✓ Validações executadas:

```

```

{validation_summary['total_checks
'}}")
    logger.logger.info(f"✓ Validações aprovadas:
{validation_summary['passed_checks
'}}")

    if not validation_summary['all_passed']:
        logger.logger.warning("⚠ Algumas validações falharam!")
        print("\n" + validator.get_validation_report())
    else:
        logger.logger.info(f"✓ Todas as validações passaram!")

# =====
# ETAPA 4: LOAD (Carga)
# =====
logger.logger.info("\n[ETAPA 4] CARGA DE DADOS (Load)")
logger.logger.info("-" * 60)

loader = DataLoader()

# 4.1 Carregar vendas enriquecidas no SQLite
loader.load_to_sqlite(
    df_enriched,
    table_name='sales_enriched',
    if_exists='replace',
    create_index=True,
    index_columns=['customer_id', 'sale_date']
)
logger.logger.info(f"✓ Vendas enriquecidas carregadas no SQLite")

# 4.2 Carregar resumo de clientes no SQLite
loader.load_to_sqlite(
    df_customer_summary,
    table_name='customer_summary',
    if_exists='replace',
    create_index=True,
    index_columns=['customer_id']
)
logger.logger.info(f"✓ Resumo de clientes carregado no SQLite")

# 4.3 Exportar para CSV (backup)
csv_path = loader.load_to_csv(
    df_enriched,
    file_name='sales_enriched.csv'
)
logger.logger.info(f"✓ Backup CSV criado: {csv_path}")

```

```

# 4.4 Exportar para Parquet (formato otimizado)
parquet_path = loader.load_to_parquet(
    df_enriched,
    file_name='sales_enriched.parquet\')
logger.logger.info(f"✓ Arquivo Parquet criado: {parquet_path}")

# 4.5 Carregar para DuckDB (para análises rápidas)
duckdb_path = str(OUTPUT_DIR / "analytics.duckdb")
loader.load_to_duckdb(
    df_enriched,
    table_name='sales_data\ ',
    db_path=duckdb_path,
    if_exists='replace\ ')
loader.load_to_duckdb(
    df_customer_summary,
    table_name='customer_summary\ ',
    db_path=duckdb_path,
    if_exists='replace\ ')
logger.logger.info(f"✓ Dados carregados no DuckDB: {duckdb_path}")

# =====
# FINALIZAÇÃO
# =====
logger.logger.info("\n" + "=" * 60)
logger.logger.info("PIPELINE ETL CONCLUÍDO COM SUCESSO! Dados
carregados em SQLite, Parquet e DuckDB. Métricas geradas.")
logger.logger.info("=" * 60)
logger.logger.info(f"Total de vendas processadas:
{len(df_enriched)}")
logger.logger.info(f"Total de clientes únicos:
{len(df_customer_summary)}")
logger.logger.info(f"Receita total: R$
{df_customer_summary['total_revenue'].sum():.2f}")

# Salvar métricas em um arquivo JSON para observabilidade
metrics = {
    "timestamp": datetime.now().isoformat(),
    "total_sales_records": len(df_enriched),
    "total_unique_customers": len(df_customer_summary),
    "total_revenue_generated":
float(df_customer_summary['total_revenue'].sum()),
    "validation_status": "Passed" if

```

```

validation_summary['all_passed'] else "Failed",
    "validation_report_summary": validation_summary['report']
}
metrics_file_path = OUTPUT_DIR / "pipeline_metrics.json"
with open(metrics_file_path, "w", encoding="utf-8") as f:
    json.dump(metrics, f, ensure_ascii=False, indent=4)
logger.logger.info(f"✓ Métricas salvas em: {metrics_file_path}")
logger.logger.info("-" * 60)
logger.logger.info("=" * 60)

return {
    'status': 'success',
    'sales_processed': len(df_enriched),
    'customers': len(df_customer_summary),
    'total_revenue':
float(df_customer_summary['total_revenue'].sum()),
    'validation_passed': validation_summary['all_passed']
}

except Exception as e:
    logger.log_error(e, "Pipeline ETL")
    logger.logger.error("=" * 60)
    logger.logger.error("PIPELINE ETL FALHOU! Verifique os logs para
mais detalhes.")
    logger.logger.error("=" * 60)
    raise

if __name__ == "__main__":
    try:
        result = run_etl_pipeline()
        print("\n✓ Pipeline executado com sucesso!")
        print(f"Resultado: {result}")
        sys.exit(0)
    except Exception as e:
        print(f"\n✗ Erro na execução do pipeline: {e}")
        sys.exit(1)

```

airflow/dags/etl_pipeline_dag.py - Orquestração Airflow

```
"""
Orquestração de Pipeline ETL com Airflow
Autor: Diego Teodoro
Data: 11/02/2026

Este DAG automatiza o processamento diário de vendas e clientes.
"""

from datetime import datetime, timedelta
from airflow import DAG
from airflow.decorators import task
import sys
from pathlib import Path

# Adicionar o diretório raiz do projeto ao path para que os módulos possam
ser importados
# Isso é uma solução para desenvolvimento local. Em produção, o projeto
seria um pacote Python instalável.
project_root = Path(__file__).resolve().parents[2] # Ajuste conforme a
estrutura do seu projeto
sys.path.insert(0, str(project_root))

# Importar os módulos do pipeline
from src.extract.extract_data import DataExtractor
from src.transform.transform_data import DataTransformer
from src.quality.data_validator import DataValidator
from src.load.load_data import DataLoader
from src.config import ETLLogger, OUTPUT_DIR

# Configuração padrão do DAG
default_args = {
    'owner': 'diego_teodoro',
    'depends_on_past': False,
    'email': ['diego.teodoro@example.com'], # Altere para seu email de
    alertas
    'email_on_failure': True,
    'email_on_retry': False,
    'retries': 3,
    'retry_delay': timedelta(minutes=5),
    'retry_exponential_backoff': True,
    'max_retry_delay': timedelta(minutes=30)
}
```

```

# Definição do DAG
with DAG(
    dag_id='etl_sales_pipeline\'',
    default_args=default_args,
    description='Pipeline ETL de vendas e clientes automatizado\'',
    schedule_interval=timedelta(days=1), # Executa diariamente
    start_date=datetime(2024, 1, 1),      # Data de início do DAG
    catchup=False,                        # Não executa para datas passadas
    max_active_runs=1, # Apenas uma execução por vez
    tags=['etl\'', \'vendas\'', \'producao\']
) as dag:

    # Documentação do DAG em português
    dag.doc_md = """
    # Pipeline ETL Automatizado de Vendas e Clientes

    Orquestração pronta para produção para processamento de conjuntos de
    dados de e-commerce.

    ## Arquitetura do Fluxo de Trabalho

    1. **Ingestão (Extração)**: Recuperação de registros de vendas e
    clientes de múltiplas fontes.
    2. **Processamento (Transformação)**: Limpeza, padronização e
    enriquecimento de conjuntos de dados brutos.
    3. **Garantia de Qualidade (Validação)**: Verificações automatizadas de
    integridade de dados.
    4. **Persistência (Carga)**: Carregamento de dados processados em SQLite
    e snapshots Parquet.
    5. **Observabilidade (Relatório)**: Geração de métricas de execução e
    relatórios de status.

    ## Metadados Operacionais

    - **Agendamento**: Diariamente às 00:00 UTC
    - **Resiliência**: 3 tentativas com backoff exponencial
    - **Alertas**: Notificações por e-mail em caso de falha
    """

    @task(task_id='extrair_dados_vendas\'')
    def extrair_vendas(**context):
        """
        Extrai dados de vendas.
        """
        logger = ETLLogger(__name__)

```

```

logger.logger.info("Iniciando extração de dados de vendas")

extractor = DataExtractor()
df_sales = extractor.extract_mock_sales_data(num_records=100)

# Salvar em XCom para próxima task
return df_sales.to_json(orient='records')

@task(task_id='extrair_dados_clientes')
def extrair_clientes(**context):
    """
    Extrai dados de clientes.
    """
    logger = ETLLoader(__name__)
    logger.logger.info("Iniciando extração de dados de clientes")

    extractor = DataExtractor()
    df_customers = extractor.extract_mock_customers_data(num_records=50)

    # Salvar em XCom para próxima task
    return df_customers.to_json(orient='records')

@task(task_id='transformar_dados')
def transformar_dados(sales_json, customers_json, **context):
    """
    Transforma e enriquece os dados.
    """
    import pandas as pd

    logger = ETLLoader(__name__)
    logger.logger.info("Iniciando transformação de dados")

    # Carregar dados do XCom
    df_sales = pd.read_json(sales_json, orient='records')
    df_customers = pd.read_json(customers_json, orient='records')

    transformer = DataTransformer()

    # Aplica as transformações completas
    df_sales = transformer.handle_missing_values(
        df_sales,
        strategy='fill',
        columns=["quantity", "unit_price", "discount"],
        fill_value=0
    )

```

```

)
df_sales = transformer.standardize_column_names(df_sales)
df_customers = transformer.standardize_column_names(df_customers)

df_sales = transformer.enforce_data_types(
    df_sales,
    {
        \'sale_id\': \'int\',
        \'customer_id\': \'int\',
        \'product_id\': \'int\',
        \'sale_date\': \'datetime\',
        \'quantity\': \'int\',
        \'unit_price\': \'float\',
        \'discount\': \'float\'
    }
)

df_customers = transformer.enforce_data_types(
    df_customers,
    {
        \'customer_id\': \'int\',
        \'registration_date\': \'datetime\'
    }
)

df_sales = transformer.add_calculated_columns(
    df_sales,
    {
        \'subtotal\': \'quantity * unit_price\',
        \'discount_amount\': \'quantity * unit_price * discount\',
        \'total_amount\': \'quantity * unit_price * (1 - discount)\'
    }
)

df_sales_completed = transformer.filter_data(
    df_sales,
    "status == \'completed\'"
)

df_sales_completed = transformer.remove_duplicates(
    df_sales_completed,
    subset=[\'sale_id\']
)

df_enriched = transformer.join_dataframes(
    df_sales_completed,

```



```

        df_customers,
        on='customer_id',
        how='left'
    )

    df_customer_summary = transformer.aggregate_data(
        df_enriched,
        group_by=['customer_id', 'first_name', 'last_name',
        'city'],
        aggregations={
            'total_amount': 'sum',
            'sale_id': 'count',
            'quantity': 'sum'
        }
    )
    df_customer_summary.columns = [
        'customer_id', 'first_name', 'last_name', 'city',
        'total_revenue', 'num_sales', 'total_items'
    ]

    return {
        'enriched': df_enriched.to_json(orient='records'),
        'summary': df_customer_summary.to_json(orient='records')
    }

```

```

@task(task_id='validar_qualidade_dados')
def validar_qualidade(transformed_data, **context):
    """
    Valida a qualidade dos dados transformados.
    """
    import pandas as pd

    logger = ETLLoader(__name__)
    logger.logger.info("Iniciando validação de qualidade")

    df_enriched = pd.read_json(
        transformed_data['enriched'],
        orient='records'
    )

    validator = DataValidator()

    validation_config = {
        "required_columns": ['sale_id', 'customer_id',
        'total_amount'],

```

```

        "value_ranges": {
            \'total_amount\': {\\'min\': 0},
            \'quantity\': {\\'min\': 1}
        }
    }

validation_summary = validator.run_all_validations(
    df_enriched,
    validation_config
)

if not validation_summary[\'all_passed\']:
    logger.logger.warning("Algumas validações falharam!")
    print(validator.get_validation_report())
    # Em um cenário real, você poderia levantar um AirflowException
    aqui para falhar a task
    # from airflow.exceptions import AirflowException
    # raise AirflowException("Falha na validação de dados")
else:
    logger.logger.info("Todas as validações passaram!")

# Retornar dados validados
return transformed_data

@task(task_id=\'carregar_dados\')
def carregar_dados(validated_data, **context):
    """
    Carrega os dados no banco de dados e em arquivos.
    """

    import pandas as pd

    logger = ETLLoader(__name__)
    logger.logger.info("Iniciando carga de dados")

    df_enriched = pd.read_json(
        validated_data[\'enriched\'],
        orient=\'records\'
    )
    df_summary = pd.read_json(
        validated_data[\'summary\'],
        orient=\'records\'
    )

    loader = DataLoader()

```

```

# Carregar vendas enriquecidas no SQLite
loader.load_to_sqlite(
    df_enriched,
    table_name='sales_enriched\\',
    if_exists='replace\\',
    create_index=True,
    index_columns=[
        'customer_id\\',
        'sale_date\\'
    ]
)

# Carregar resumo de clientes no SQLite
loader.load_to_sqlite(
    df_summary,
    table_name='customer_summary\\',
    if_exists='replace\\',
    create_index=True,
    index_columns=[
        'customer_id\\'
    ]
)

# Exportar backup CSV
loader.load_to_csv(df_enriched, file_name='sales_enriched.csv\\')

# Exportar para Parquet (formato otimizado)
loader.load_to_parquet(df_enriched,
file_name='sales_enriched.parquet\\')

# Carregar para DuckDB (para análises rápidas)
duckdb_path = str(OUTPUT_DIR / "analytics.duckdb")
loader.load_to_duckdb(
    df_enriched,
    table_name='sales_data\\',
    db_path=duckdb_path,
    if_exists='replace\\'
)
loader.load_to_duckdb(
    df_summary,
    table_name='customer_summary\\',
    db_path=duckdb_path,
    if_exists='replace\\'
)

return {

```

```

        \ 'sales_count\': len(df_enriched),
        \ 'customers_count\': len(df_summary),
        \ 'total_revenue\': float(df_summary[\ 'total_revenue\'].sum())
    }

@task(task_id=\ 'gerar_relatorio_final\')
def gerar_relatorio(load_result, **context):
    """
    Gera relatório de execução do pipeline e métricas.
    """
    import json
    from datetime import datetime

    logger = ETLLogger(__name__)

    execution_date = context[\ 'ds\']

    # Carregar métricas do arquivo JSON gerado pelo main.py
    metrics_file_path = OUTPUT_DIR / "pipeline_metrics.json"
    metrics = {}
    try:
        with open(metrics_file_path, "r", encoding="utf-8") as f:
            metrics = json.load(f)
        logger.logger.info(f"Métricas carregadas de:
{metrics_file_path}")
    except FileNotFoundError:
        logger.logger.error(f"Arquivo de métricas não encontrado:
{metrics_file_path}")
        metrics = {
            "total_sales_records": load_result[\ 'sales_count\'],
            "total_unique_customers": load_result[\ 'customers_count\'],
            "total_revenue_generated": load_result[\ 'total_revenue\'],
            "validation_status": "Unknown"
        }

    report = f"""
=====
RELATÓRIO DE EXECUÇÃO DO PIPELINE ETL
=====
Data de Execução: {execution_date}
Timestamp da Geração: {metrics.get("timestamp", "N/A")}

Resultados:
- Vendas processadas: {metrics.get("total_sales_records", "N/A")}
- Clientes únicos: {metrics.get("total_unique_customers", "N/A")}

```

```

- Receita total: R$ {metrics.get("total_revenue_generated", 0):.2f}

Status Validação: {metrics.get("validation_status", "N/A")}
Status Geral: SUCESSO
=====
"""

logger.logger.info(report)
print(report)

return load_result

# Definir fluxo de execução (dependências)
sales_data = extrair_vendas()
customers_data = extrair_clientes()

transformed = transformar_dados(sales_data, customers_data)
validated = validar_qualidade(transformed)
loaded = carregar_dados(validated)
report = gerar_relatorio(loaded)

# Documentação das tasks
extrair_vendas.doc_md = "Extrai dados de vendas da fonte de dados simulada."
extrair_clientes.doc_md = "Extrai dados de clientes da fonte de dados simulada."
transformar_dados.doc_md = "Aplica transformações, limpeza e enriquecimento nos dados extraídos."
validar_qualidade.doc_md = "Verifica a integridade e a qualidade dos dados transformados."
carregar_dados.doc_md = "Persiste os dados processados em SQLite, CSV e Parquet."
gerar_relatorio.doc_md = "Cria um relatório final com as métricas de execução do pipeline."

```

src/extract/extract_data.py - Módulo de Extração

```
"""
```

```
Extração de Dados (Extract)
```

```
Autor: Diego Teodoro
```

```
Este módulo contém a lógica necessária para coletar dados de fontes  
variadas,  
como APIs, arquivos locais e bancos de dados.
```

```
"""
```

```
import pandas as pd
```

```
import requests
```

```
import time
```

```
from typing import Dict, Any, Optional
```

```
from pathlib import Path
```

```
from ..config import ETLLogger, RETRY_CONFIG, INPUT_DIR
```

```
class DataExtractor:
```

```
    """
```

```
    Classe responsável por extrair dados de diversas fontes.
```

```
    """
```

```
    def __init__(self):
```

```
        self.logger = ETLLogger(__name__)
```

```
    def extract_from_api(
```

```
        self,
```

```
        url: str,
```

```
        method: str = 'GET',
```

```
        headers: Optional[Dict[str, str]] = None,
```

```
        params: Optional[Dict[str, Any]] = None,
```

```
        timeout: int = 30
```

```
) -> Dict[str, Any]:
```

```
    """
```

```
    Extrai dados de uma API REST com retry automático.
```

```
    Args:
```

```
        url: URL da API
```

```
        method: Método HTTP (GET, POST, etc.)
```

```
        headers: Headers da requisição
```

```
        params: Parâmetros da requisição
```

```
        timeout: Timeout em segundos
```

```
    Returns:
```

```

        Dicionário com os dados extraídos
    """
    self.logger.log_extraction_start("API", url=url, method=method)

    max_attempts = RETRY_CONFIG["max_attempts"]
    delay = RETRY_CONFIG["initial_delay"]

    for attempt in range(1, max_attempts + 1):
        try:
            response = requests.request(
                method=method,
                url=url,
                headers=headers,
                params=params,
                timeout=timeout
            )
            response.raise_for_status()

            data = response.json()
            record_count = len(data) if isinstance(data, list) else 1

            self.logger.log_extraction_complete(
                "API",
                records=record_count,
                url=url,
                status_code=response.status_code
            )

            return data

        except requests.exceptions.Timeout:
            self.logger.logger.warning(
                f"Timeout na tentativa {attempt}/{max_attempts} - URL:
{url}"
            )
            if attempt < max_attempts:
                time.sleep(delay)
                delay *= RETRY_CONFIG["backoff_multiplier"]
            else:
                raise

        except requests.exceptions.RequestException as e:
            self.logger.log_error(e, f"Extração de API (tentativa
{attempt})")
            if attempt < max_attempts:
                time.sleep(delay)

```

```

        delay *= RETRY_CONFIG["backoff_multiplier"]
    else:
        raise

def extract_from_csv(
    self,
    file_path: str,
    encoding: str = 'utf-8\\',
    delimiter: str = '\\',\\',
    **kwargs
) -> pd.DataFrame:
    """
    Extrai dados de um arquivo CSV.

    Args:
        file_path: Caminho do arquivo CSV
        encoding: Codificação do arquivo
        delimiter: Delimitador usado no CSV
        **kwargs: Argumentos adicionais para pd.read_csv

    Returns:
        DataFrame com os dados extraídos
    """
    self.logger.log_extraction_start("CSV", file_path=file_path)

    try:
        # Se o caminho for relativo, usar INPUT_DIR como base
        path = Path(file_path)
        if not path.is_absolute():
            path = INPUT_DIR / file_path

        df = pd.read_csv(
            path,
            encoding=encoding,
            delimiter=delimiter,
            **kwargs
        )

        self.logger.log_extraction_complete(
            "CSV",
            records=len(df),
            file_path=str(path),
            columns=list(df.columns)
        )

    return df

```



```

        except FileNotFoundError as e:
            self.logger.log_error(e, "Extração de CSV - Arquivo não
encontrado")
            raise
        except Exception as e:
            self.logger.log_error(e, "Extração de CSV")
            raise

def extract_from_json(
    self,
    file_path: str,
    encoding: str = '\utf-8\ ',
    **kwargs
) -> pd.DataFrame:
    """
    Extrai dados de um arquivo JSON.

    Args:
        file_path: Caminho do arquivo JSON
        encoding: Codificação do arquivo
        **kwargs: Argumentos adicionais para pd.read_json

    Returns:
        DataFrame com os dados extraídos
    """
    self.logger.log_extraction_start("JSON", file_path=file_path)

    try:
        # Se o caminho for relativo, usar INPUT_DIR como base
        path = Path(file_path)
        if not path.is_absolute():
            path = INPUT_DIR / file_path

        df = pd.read_json(
            path,
            encoding=encoding,
            **kwargs
        )

        self.logger.log_extraction_complete(
            "JSON",
            records=len(df),
            file_path=str(path),
            columns=list(df.columns)
        )

```

```

        return df

    except FileNotFoundError as e:
        self.logger.log_error(e, "Extração de JSON - Arquivo não
encontrado")
        raise
    except Exception as e:
        self.logger.log_error(e, "Extração de JSON")
        raise

def extract_from_parquet(
    self,
    file_path: str,
    **kwargs
) -> pd.DataFrame:
    """
    Extrai dados de um arquivo Parquet.

    Args:
        file_path: Caminho do arquivo Parquet
        **kwargs: Argumentos adicionais para pd.read_parquet

    Returns:
        DataFrame com os dados extraídos
    """
    self.logger.log_extraction_start("Parquet", file_path=file_path)

    try:
        # Se o caminho for relativo, usar INPUT_DIR como base
        path = Path(file_path)
        if not path.is_absolute():
            path = INPUT_DIR / file_path

        df = pd.read_parquet(path, **kwargs)

        self.logger.log_extraction_complete(
            "Parquet",
            records=len(df),
            file_path=str(path),
            columns=list(df.columns)
        )

        return df

    except FileNotFoundError as e:

```

```

        self.logger.log_error(e, "Extração de Parquet - Arquivo não
encontrado")
        raise
    except Exception as e:
        self.logger.log_error(e, "Extração de Parquet")
        raise

    def extract_mock_sales_data(self, num_records: int = 100) ->
pd.DataFrame:
    """
    Geração de dados sintéticos de vendas para validação do pipeline.

    Args:
        num_records: Quantidade de registros a serem gerados.

    Returns:
        pd.DataFrame: Dados de vendas gerados.
    """
    import numpy as np
    from datetime import datetime, timedelta

    self.logger.log_extraction_start("Mock Sales Data",
num_records=num_records)

    # Gerar dados simulados
    np.random.seed(42)

    start_date = datetime(2024, 1, 1)
    dates = [start_date + timedelta(days=x) for x in range(num_records)]

    df = pd.DataFrame({
        \'sale_id\': range(1, num_records + 1),
        \'customer_id\': np.random.randint(1, 51, num_records),
        \'product_id\': np.random.randint(1, 21, num_records),
        \'sale_date\': dates,
        \'quantity\': np.random.randint(1, 10, num_records),
        \'unit_price\': np.round(np.random.uniform(10, 500,
num_records), 2),
        \'discount\': np.round(np.random.uniform(0, 0.3, num_records),
2),
        \'status\': np.random.choice([\'completed\', \'pending\',
\'cancelled\'], num_records, p=[0.8, 0.15, 0.05])
    })

    self.logger.log_extraction_complete("Mock Sales Data",
records=len(df))

```

```

        return df

    def extract_mock_customers_data(self, num_records: int = 50) ->
pd.DataFrame:
    """
    Geração de dados sintéticos de clientes para validação do pipeline.

    Args:
        num_records: Quantidade de registros a serem gerados.

    Returns:
        pd.DataFrame: Dados de clientes gerados.
    """
    import numpy as np

    self.logger.log_extraction_start("Mock Customers Data",
num_records=num_records)

    # Gerar dados simulados
    np.random.seed(42)

    first_names = ['João', 'Maria', 'Pedro', 'Ana', 'Carlos',
'Juliana', 'Lucas', 'Fernanda']
    last_names = ['Silva', 'Santos', 'Oliveira', 'Souza',
'Lima', 'Costa', 'Ferreira', 'Rodrigues']

    df = pd.DataFrame({
        'customer_id': range(1, num_records + 1),
        'first_name': np.random.choice(first_names, num_records),
        'last_name': np.random.choice(last_names, num_records),
        'email': [f"cliente{i}@email.com" for i in range(1,
num_records + 1)],
        'city': np.random.choice(['São Paulo', 'Rio de Janeiro',
'Belo Horizonte', 'Brasília', 'Curitiba'], num_records),
        'state': np.random.choice(['SP', 'RJ', 'MG', 'DF',
'PR'], num_records),
        'registration_date': pd.date_range(start='2023-01-01',
periods=num_records, freq='W')
    })

    self.logger.log_extraction_complete("Mock Customers Data",
records=len(df))

    return df

```


src/transform/transform_data.py - Módulo de Transformação

```
"""
```

```
Transformação de Dados (Transform)
```

```
Autor: Diego Teodoro
```

```
Neste módulo, implemento as regras de negócio, limpeza e enriquecimento dos dados brutos coletados.
```

```
"""
```

```
import pandas as pd
```

```
import numpy as np
```

```
from typing import List, Dict, Any, Optional
```

```
from datetime import datetime
```

```
from ..config import ETLLogger
```

```
class DataTransformer:
```

```
    """
```

```
    Classe responsável por aplicar transformações nos dados.
```

```
    """
```

```
    def __init__(self):
```

```
        self.logger = ETLLogger(__name__)
```

```
    def standardize_column_names(self, df: pd.DataFrame) -> pd.DataFrame:
```

```
        """
```

```
        Padroniza os nomes das colunas para minúsculas e substitui espaços por underscores.
```

```
        Args:
```

```
            df: DataFrame de entrada.
```

```
        Returns:
```

```
            DataFrame com nomes de colunas padronizados.
```

```
        """
```

```
        self.logger.log_transformation_start("Padronização de nomes de colunas")
```

```
        original_columns = df.columns.tolist()
```

```
        df.columns = df.columns.str.lower().str.replace(" ", "_")
```

```
        self.logger.log_transformation_complete(
```

```
            "Padronização de nomes de colunas",
```

```
            records_in=len(df),
```

```
            records_out=len(df),
```

```
            original_columns=original_columns,
```

```
            new_columns=df.columns.tolist()
```

```

    )
    return df

    def enforce_data_types(self, df: pd.DataFrame, type_mapping: Dict[str,
str]) -> pd.DataFrame:
        """
        Garante que as colunas tenham os tipos de dados corretos.

        Args:
            df: DataFrame de entrada.
            type_mapping: Dicionário com {nome_coluna: tipo_desejado}.

        Returns:
            DataFrame com tipos de dados corrigidos.
        """
        self.logger.log_transformation_start("Aplicação de tipos de dados",
type_mapping=type_mapping)
        for col, dtype in type_mapping.items():
            if col in df.columns:
                try:
                    if dtype == \'datetime\':
                        df[col] = pd.to_datetime(df[col], errors=\'coerce\')
                    else:
                        df[col] = df[col].astype(dtype)
                except Exception as e:
                    self.logger.log_error(e, f"Erro ao converter coluna
{col} para {dtype}")
        self.logger.log_transformation_complete(
            "Aplicação de tipos de dados",
            records_in=len(df),
            records_out=len(df)
        )
        return df

    def add_calculated_columns(self, df: pd.DataFrame, calculations:
Dict[str, str]) -> pd.DataFrame:
        """
        Adiciona novas colunas baseadas em cálculos de colunas existentes.

        Args:
            df: DataFrame de entrada.
            calculations: Dicionário com {nova_coluna:
expressão_de_cálculo}.

        Returns:
            DataFrame com as novas colunas.

```

```

        """
        self.logger.log_transformation_start("Adição de colunas calculadas",
calculations=calculations)
        for new_col, expression in calculations.items():
            try:
                df[new_col] = df.eval(expression)
            except Exception as e:
                self.logger.log_error(e, f"Erro ao calcular coluna {new_col}
com expressão \\'{expression}\\'")
            self.logger.log_transformation_complete(
                "Adição de colunas calculadas",
                records_in=len(df),
                records_out=len(df)
            )
        return df

def filter_data(self, df: pd.DataFrame, condition: str) -> pd.DataFrame:
    """
    Filtra o DataFrame com base em uma condição.

    Args:
        df: DataFrame de entrada.
        condition: Expressão de filtro (ex: "column > 10").

    Returns:
        DataFrame filtrado.
    """
    self.logger.log_transformation_start("Filtragem de dados",
condition=condition)
    records_in = len(df)
    try:
        df_filtered = df.query(condition)
    except Exception as e:
        self.logger.log_error(e, f"Erro ao filtrar dados com condição
\\'{condition}\\'")
        df_filtered = df # Retorna o original em caso de erro
    records_out = len(df_filtered)
    self.logger.log_transformation_complete(
        "Filtragem de dados",
        records_in=records_in,
        records_out=records_out,
        condition=condition
    )
    return df_filtered

def remove_duplicates(self, df: pd.DataFrame, subset:

```



```

Optional[List[str]] = None) -> pd.DataFrame:
    """
    Remove linhas duplicadas do DataFrame.

    Args:
        df: DataFrame de entrada.
        subset: Lista de colunas para considerar na identificação de
duplicatas.

    Returns:
        DataFrame sem duplicatas.
    """
    self.logger.log_transformation_start("Remoção de duplicatas",
subset=subset)
    records_in = len(df)
    df_deduplicated = df.drop_duplicates(subset=subset)
    records_out = len(df_deduplicated)
    self.logger.log_transformation_complete(
        "Remoção de duplicatas",
        records_in=records_in,
        records_out=records_out,
        duplicates_removed=records_in - records_out
    )
    return df_deduplicated


def handle_missing_values(self, df: pd.DataFrame, strategy: str =
\'drop\', columns: Optional[List[str]] = None, fill_value: Any = None) ->
pd.DataFrame:
    """
    Trata valores ausentes (NaN).

    Args:
        df: DataFrame de entrada.
        strategy: Estratégia (\'drop\', \'fill\').
        columns: Lista de colunas para aplicar a estratégia. Se None,
aplica a todas.
        fill_value: Valor para preencher se strategy=\'fill\'.

    Returns:
        DataFrame com valores ausentes tratados.
    """
    self.logger.log_transformation_start("Tratamento de valores
ausentes", strategy=strategy, columns=columns)
    records_in = len(df)
    if strategy == \'drop\':
        df_processed = df.dropna(subset=columns)

```

```

elif strategy == \'fill\' and fill_value is not None:
    if columns:
        df_processed = df.fillna(dict.fromkeys(columns, fill_value))
    else:
        df_processed = df.fillna(fill_value)
else:
    self.logger.logger.warning(f"Estratégia de tratamento de nulos
inválida ou incompleta: {strategy}")
    df_processed = df
records_out = len(df_processed)
self.logger.log_transformation_complete(
    "Tratamento de valores ausentes",
    records_in=records_in,
    records_out=records_out,
    strategy=strategy,
    missing_values_handled=records_in - records_out
)
return df_processed

def join_dataframes(self, left_df: pd.DataFrame, right_df: pd.DataFrame,
on: str, how: str = \'inner\') -> pd.DataFrame:
    """
    Realiza um join entre dois DataFrames.

    Args:
        left_df: DataFrame da esquerda.
        right_df: DataFrame da direita.
        on: Coluna para realizar o join.
        how: Tipo de join (\'inner\', \'left\', \'right\', \'outer\').

    Returns:
        DataFrame resultante do join.
    """
    self.logger.log_transformation_start("Join de DataFrames", on=on,
how=how)
    records_in_left = len(left_df)
    records_in_right = len(right_df)
    df_joined = pd.merge(left_df, right_df, on=on, how=how)
    records_out = len(df_joined)
    self.logger.log_transformation_complete(
        "Join de DataFrames",
        records_in=f"left:{records_in_left}, right:{records_in_right}",
        records_out=records_out,
        on_column=on,
        join_type=how
    )

```

```

        return df_joined

    def aggregate_data(self, df: pd.DataFrame, group_by: List[str],
aggregations: Dict[str, str]) -> pd.DataFrame:
        """
        Agrega dados com base em colunas e funções de agregação.

        Args:
            df: DataFrame de entrada.
            group_by: Lista de colunas para agrupar.
            aggregations: Dicionário com {coluna_para_agregar:
função_de_agregação}.

        Returns:
            DataFrame agregado.
        """
        self.logger.log_transformation_start("Agregação de dados",
group_by=group_by, aggregations=aggregations)
        records_in = len(df)
        df_aggregated = df.groupby(group_by).agg(aggregations).reset_index()
        records_out = len(df_aggregated)
        self.logger.log_transformation_complete(
            "Agregação de dados",
            records_in=records_in,
            records_out=records_out,
            group_by_columns=group_by
        )
        return df_aggregated

```

src/quality/data_validator.py - Motor de Qualidade

```
"""
Qualidade de Dados (Data Quality)
Autor: Diego Teodoro

Este módulo é responsável por garantir que os dados processados atendam
aos critérios de qualidade e integridade definidos.
"""

import pandas as pd
from typing import List, Dict, Any, Optional
from datetime import datetime
from ..config import ETLLogger, VALIDATION_CONFIG


class DataValidator:
    """
    Classe responsável por realizar validações de qualidade nos dados.
    """

    def __init__(self):
        self.logger = ETLLogger(__name__)
        self.validation_report = []

    def _log_validation_result(self, check_name: str, status: str, details: str):
        self.validation_report.append({
            "check_name": check_name,
            "status": status,
            "details": details
        })
        if status == "FAILED":
            self.logger.log_validation_issue(check_name, details)
        else:
            self.logger.logger.info(f"Validação \"{check_name}\" {status}: {details}")

    def check_required_columns(self, df: pd.DataFrame, required_columns: List[str]) -> bool:
        """
        Verifica se todas as colunas obrigatórias estão presentes.
        """
        missing_columns = [col for col in required_columns if col not in df.columns]
        if missing_columns:
```

```

        self._log_validation_result(
            "Required Columns Check",
            "FAILED",
            f"Colunas obrigatórias ausentes: {\', \'
\'.join(missing_columns)}"
        )
        return False
    self._log_validation_result(
        "Required Columns Check",
        "PASSED",
        "Todas as colunas obrigatórias estão presentes."
    )
    return True

def check_no_nulls_in_columns(self, df: pd.DataFrame, columns:
List[str]) -> bool:
    """
    Verifica se não há valores nulos em colunas específicas.
    """
    null_columns = df[columns].isnull().any()
    if null_columns.any():
        cols_with_nulls = null_columns[null_columns ==
True].index.tolist()
        self._log_validation_result(
            "No Nulls Check",
            "FAILED",
            f"Colunas com valores nulos: {\', \'
\'.join(cols_with_nulls)}"
        )
        return False
    self._log_validation_result(
        "No Nulls Check",
        "PASSED",
        "Nenhuma coluna especificada contém valores nulos."
    )
    return True

def check_unique_values(self, df: pd.DataFrame, column: str) -> bool:
    """
    Verifica se os valores em uma coluna são únicos.
    """
    if not df[column].is_unique:
        duplicate_count = df[column].duplicated().sum()
        self._log_validation_result(
            "Unique Values Check",
            "FAILED",
            f"Coluna \"{column}\" contém {duplicate_count} valores

```

```

duplicados."
    )
    return False
self._log_validation_result(
    "Unique Values Check",
    "PASSED",
    f"Coluna \"{column}\" contém apenas valores únicos."
)
return True

def check_value_range(self, df: pd.DataFrame, column: str, min_val:
Optional[Any] = None, max_val: Optional[Any] = None) -> bool:
    """
    Verifica se os valores em uma coluna estão dentro de um range.
    """
    out_of_range = pd.Series([False] * len(df))
    details = []
    if min_val is not None:
        out_of_range = out_of_range | (df[column] < min_val)
        details.append(f"Min: {min_val}")
    if max_val is not None:
        out_of_range = out_of_range | (df[column] > max_val)
        details.append(f"Max: {max_val}")

    if out_of_range.any():
        self._log_validation_result(
            "Value Range Check",
            "FAILED",
            f"Coluna \"{column}\" contém {out_of_range.sum()} valores
fora do range ({{'\ ', '\ '.join(details)}})."
        )
        return False
    self._log_validation_result(
        "Value Range Check",
        "PASSED",
        f"Todos os valores da coluna \"{column}\" estão dentro do
range ({{'\ ', '\ '.join(details)}})."
    )
    return True

def check_data_type_conformance(self, df: pd.DataFrame, column: str,
expected_type: type) -> bool:
    """
    Verifica se uma coluna está em conformidade com o tipo de dado
esperado.
    """

```

```

        # Pandas pode representar inteiros com NaN como float, então
verificamos o tipo base
        if expected_type == int:
            is_conforming = pd.api.types.is_integer_dtype(df[column]) or \
                (pd.api.types.is_float_dtype(df[column]) and
(df[column] == df[column].astype(int)).all())
        elif expected_type == float:
            is_conforming = pd.api.types.is_float_dtype(df[column])
        elif expected_type == str:
            is_conforming = pd.api.types.is_string_dtype(df[column])
        elif expected_type == datetime:
            is_conforming = pd.api.types.is_datetime64_any_dtype(df[column])
        else:
            is_conforming = False # Tipo não suportado para checagem
específica

        if not is_conforming:
            self._log_validation_result(
                "Data Type Conformance Check",
                "FAILED",
                f"Coluna \"{column}\" não está em conformidade com o tipo
esperado {expected_type.__name__}."
            )
            return False
        self._log_validation_result(
            "Data Type Conformance Check",
            "PASSED",
            f"Coluna \"{column}\" está em conformidade com o tipo esperado
{expected_type.__name__}."
        )
        return True

    def run_all_validations(self, df: pd.DataFrame, config: Dict[str, Any])
-> Dict[str, Any]:
        """
        Executa um conjunto de validações com base em um dicionário de
configuração.

        Args:
            df: DataFrame a ser validado.
            config: Dicionário de configuração das validações.
            Exemplo:
            {
                "required_columns": ["col1", "col2"],
                "no_null_columns": ["col1"],
                "unique_columns": ["col1"],

```

```
        "value_ranges": {"col2": {"min": 0, "max": 100}},
        "expected_types": {"col1": int, "col2": float}
    }
```

Returns:

Um dicionário resumindo os resultados das validações.

```
"""
    self.logger.log_transformation_start("Executando todas as
validações", config=config)
    self.validation_report = [] # Resetar o relatório para cada execução
    all_passed = True
    total_checks = 0
    passed_checks = 0

    # 1. Required Columns
    if "required_columns" in config:
        total_checks += 1
        if self.check_required_columns(df, config["required_columns"]):
            passed_checks += 1

    # 2. No Nulls in Columns
    if "no_null_columns" in config:
        total_checks += 1
        if self.check_no_nulls_in_columns(df,
config["no_null_columns"]):
            passed_checks += 1

    # 3. Unique Columns
    if "unique_columns" in config:
        for col in config["unique_columns"]:
            total_checks += 1
            if self.check_unique_values(df, col):
                passed_checks += 1

    # 4. Value Ranges
    if "value_ranges" in config:
        for col, limits in config["value_ranges"].items():
            total_checks += 1
            if self.check_value_range(df, col, limits.get("min"),
limits.get("max")):
                passed_checks += 1

    # 5. Data Type Conformance
    if "expected_types" in config:
        for col, expected_type in config["expected_types"].items():
            total_checks += 1
```



```

        if self.check_data_type_conformance(df, col, expected_type):
            passed_checks += 1

    all_passed = (total_checks == passed_checks)
    self.logger.log_transformation_complete(
        "Execução de todas as validações",
        records_in=len(df),
        records_out=len(df),
        all_passed=all_passed,
        total_checks=total_checks,
        passed_checks=passed_checks
    )
    return {
        "all_passed": all_passed,
        "total_checks": total_checks,
        "passed_checks": passed_checks,
        "report": self.validation_report
    }

def get_validation_report(self) -> str:
    """
    Retorna o relatório de validação formatado.
    """
    report_str = "\n--- Relatório de Validação de Dados ---\n"
    for entry in self.validation_report:
        report_str += f"- [{entry['status']}] {entry['check_name']}: "
        {entry['details']}\n"
        report_str += "-----\n"
    return report_str

# Exemplo de validações específicas para o pipeline de vendas
def validate_sales_data(self, df: pd.DataFrame) -> bool:
    """
    Valida o DataFrame de vendas enriquecido.
    """
    self.logger.log_transformation_start("Validação de dados de vendas")
    config = {
        "required_columns": ["sale_id", "customer_id", "product_id",
"sale_date", "total_amount"],
        "no_null_columns": ["sale_id", "customer_id", "total_amount"],
        "unique_columns": ["sale_id"],
        "value_ranges": {
            "quantity": {"min": 1},
            "unit_price": {"min": 0.01},
            "total_amount": {"min": 0}
        },
    },

```

```

        "expected_types": {
            "sale_id": int,
            "customer_id": int,
            "product_id": int,
            "sale_date": datetime,
            "quantity": int,
            "unit_price": float,
            "discount": float,
            "total_amount": float
        }
    }
    result = self.run_all_validations(df, config)
    self.logger.log_transformation_complete("Validação de dados de vendas", len(df), len(df), all_passed=result["all_passed"])
    return result["all_passed"]

def validate_customer_summary(self, df: pd.DataFrame) -> bool:
    """
    Valida o DataFrame de sumário de clientes.
    """
    self.logger.log_transformation_start("Validação de sumário de clientes")
    config = {
        "required_columns": ["customer_id", "total_revenue", "num_sales"],
        "no_null_columns": ["customer_id", "total_revenue"],
        "unique_columns": ["customer_id"],
        "value_ranges": {
            "total_revenue": {"min": 0},
            "num_sales": {"min": 0}
        },
        "expected_types": {
            "customer_id": int,
            "total_revenue": float,
            "num_sales": int
        }
    }
    result = self.run_all_validations(df, config)
    self.logger.log_transformation_complete("Validação de sumário de clientes", len(df), len(df), all_passed=result["all_passed"])
    return result["all_passed"]

```

src/load/load_data.py - Módulo de Carga

```
"""
```

```
Carga de Dados (Load)
```

```
Autor: Diego Teodoro
```

```
Este módulo gerencia a persistência dos dados transformados em seus  
destinos finais, como bancos de dados e arquivos estruturados.
```

```
"""
```

```
import pandas as pd
```

```
from typing import Optional, Dict, Any, List
```

```
from pathlib import Path
```

```
from ..config import ETLLogger, OUTPUT_DIR
```

```
from ..db_connection.sqlite_connector import SQLiteConnector
```

```
class DataLoader:
```

```
    """
```

```
    Classe responsável por carregar dados em diferentes destinos.
```

```
    """
```

```
    def __init__(self):
```

```
        self.logger = ETLLogger(__name__)
```

```
    def load_to_sqlite(
```

```
        self,
```

```
        df: pd.DataFrame,
```

```
        table_name: str,
```

```
        db_path: Optional[str] = None,
```

```
        if_exists: str = 'append',
```

```
        create_index: bool = False,
```

```
        index_columns: Optional[List[str]] = None
```

```
) -> int:
```

```
    """
```

```
    Carrega dados em uma tabela SQLite.
```

```
    Args:
```

```
        df: DataFrame a ser carregado
```

```
        table_name: Nome da tabela de destino
```

```
        db_path: Caminho do banco de dados (None = usar padrão)
```

```
        if_exists: Comportamento se tabela existir ('fail',  
'replace', 'append')
```

```
        create_index: Se True, cria índice nas colunas especificadas
```

```
        index_columns: Colunas para criar índice
```

```

Returns:
    Número de registros carregados
    """
self.logger.log_load_start(
    f"SQLite.{table_name}",
    records=len(df),
    if_exists=if_exists
)

try:
    with SQLiteConnector(db_path) as connector:
        # Carregar dados
        rows_written = connector.write_dataframe(
            df,
            table_name,
            if_exists=if_exists
        )

        # Criar índice se solicitado
        if create_index and index_columns:
            self._create_index(connector, table_name, index_columns)

        self.logger.log_load_complete(
            f"SQLite.{table_name}",
            records=rows_written
        )

        return rows_written

except Exception as e:
    self.logger.log_error(e, f"Carga em SQLite.{table_name}")
    raise

def load_to_duckdb(
    self,
    df: pd.DataFrame,
    table_name: str,
    db_path: Optional[str] = None,
    if_exists: str = '\append\ ',
    **kwargs
) -> int:
    """
    Carrega um DataFrame para uma tabela DuckDB.

    Args:
        df: DataFrame a ser carregado.

```

```
        table_name: Nome da tabela no DuckDB.
        db_path: Caminho para o arquivo do banco de dados DuckDB (se
None, usa \':memory:\').
        if_exists: Comportamento se a tabela já existe (\'fail\',
\'replace\', \'append\').
        **kwargs: Argumentos adicionais para df.to_sql.
```

```
Returns:
    Número de registros carregados.
"""
import duckdb

self.logger.log_load_start(
    f"DuckDB.{table_name}",
    records=len(df),
    if_exists=if_exists
)

try:
    conn = duckdb.connect(database=db_path or \':memory:\')
    rows_written = df.to_sql(
        table_name,
        conn,
        if_exists=if_exists,
        index=False, # DuckDB não suporta index do pandas
diretamente
        **kwargs
    )
    conn.close()

    self.logger.log_load_complete(
        f"DuckDB.{table_name}",
        records=rows_written
    )

    return rows_written

except Exception as e:
    self.logger.log_error(e, f"Carga em DuckDB.{table_name}")
    raise

def _create_index(
    self,
    connector: SQLiteConnector,
    table_name: str,
    columns: list
```

```

):
    """Cria índice em colunas específicas."""
    index_name = f"idx_{table_name}_{'\_\''.join(columns)}"
    columns_str = ", ".join(columns)

    query = f"""
        CREATE INDEX IF NOT EXISTS {index_name}
        ON {table_name} ({columns_str})
    """

    try:
        connector.execute_query(query)
        connector.connection.commit()
        self.logger.logger.info(f"Índice criado: {index_name}")
    except Exception as e:
        self.logger.logger.warning(f"Erro ao criar índice: {e}")

def load_to_csv(
    self,
    df: pd.DataFrame,
    file_name: str,
    output_dir: Optional[Path] = None,
    encoding: str = '\utf-8\ ',
    index: bool = False,
    **kwargs
) -> str:
    """
    Carrega dados em um arquivo CSV.

    Args:
        df: DataFrame a ser carregado
        file_name: Nome do arquivo de saída
        output_dir: Diretório de saída (None = usar padrão)
        encoding: Codificação do arquivo
        index: Se True, escreve o índice do DataFrame
        **kwargs: Argumentos adicionais para to_csv

    Returns:
        Caminho completo do arquivo criado
    """
    output_path = (output_dir or OUTPUT_DIR) / file_name

    self.logger.log_load_start(
        f"CSV: {file_name}",
        records=len(df)
    )

```

```

try:
    df.to_csv(
        output_path,
        encoding=encoding,
        index=index,
        **kwargs
    )

    self.logger.log_load_complete(
        f"CSV: {file_name}",
        records=len(df),
        file_path=str(output_path)
    )

    return str(output_path)

except Exception as e:
    self.logger.log_error(e, f"Carga em CSV: {file_name}")
    raise

def load_to_parquet(
    self,
    df: pd.DataFrame,
    file_name: str,
    output_dir: Optional[Path] = None,
    compression: str = '\snappy\ ',
    **kwargs
) -> str:
    """
    Carrega dados em um arquivo Parquet.

    Args:
        df: DataFrame a ser carregado
        file_name: Nome do arquivo de saída
        output_dir: Diretório de saída (None = usar padrão)
        compression: Tipo de compressão ('\snappy\ ', '\gzip\ ',
        '\brotli\ ', None)
        **kwargs: Argumentos adicionais para to_parquet

    Returns:
        Caminho completo do arquivo criado
    """
    output_path = (output_dir or OUTPUT_DIR) / file_name

    self.logger.log_load_start(

```

```

        f"Parquet: {file_name}",
        records=len(df)
    )

    try:
        df.to_parquet(
            output_path,
            compression=compression,
            **kwargs
        )

        self.logger.log_load_complete(
            f"Parquet: {file_name}",
            records=len(df),
            file_path=str(output_path)
        )

        return str(output_path)

    except Exception as e:
        self.logger.log_error(e, f"Carga em Parquet: {file_name}")
        raise

def load_to_json(
    self,
    df: pd.DataFrame,
    file_name: str,
    output_dir: Optional[Path] = None,
    orient: str = '\records\' ,
    **kwargs
) -> str:
    """
    Carrega dados em um arquivo JSON.

    Args:
        df: DataFrame a ser carregado
        file_name: Nome do arquivo de saída
        output_dir: Diretório de saída (None = usar padrão)
        orient: Formato do JSON ('\records\' , '\index\' , '\columns\' ,
etc.)

        **kwargs: Argumentos adicionais para to_json

    Returns:
        Caminho completo do arquivo criado
    """
    output_path = (output_dir or OUTPUT_DIR) / file_name

```



```

self.logger.log_load_start(
    f"JSON: {file_name}",
    records=len(df)
)

try:
    df.to_json(
        output_path,
        orient=orient,
        **kwargs
    )

    self.logger.log_load_complete(
        f"JSON: {file_name}",
        records=len(df),
        file_path=str(output_path)
    )

    return str(output_path)

except Exception as e:
    self.logger.log_error(e, f"Carga em JSON: {file_name}")
    raise

def load_incremental(
    self,
    df: pd.DataFrame,
    table_name: str,
    key_column: str,
    db_path: Optional[str] = None
) -> Dict[str, int]:
    """
    Realiza carga incremental baseada em uma chave.
    Atualiza registros existentes e insere novos.

    Args:
        df: DataFrame com dados novos
        table_name: Nome da tabela
        key_column: Coluna chave para identificar registros
        db_path: Caminho do banco de dados

    Returns:
        Dicionário com contagem de inserções e atualizações
    """
    self.logger.log_load_start(

```

```

        f"SQLite.{table_name} (incremental)",
        records=len(df),
        key_column=key_column
    )

try:
    with SQLiteConnector(db_path) as connector:
        # Verificar se tabela existe
        if not connector.table_exists(table_name):
            # Se não existe, fazer carga completa
            rows_written = connector.write_dataframe(
                df,
                table_name,
                if_exists='replace'
            )

            result = {
                '\inserted\': rows_written,
                '\updated\': 0,
                '\total\': rows_written
            }
        else:
            # Ler dados existentes
            existing_df = connector.read_to_dataframe(
                f"SELECT * FROM {table_name}"
            )

            # Identificar novos e existentes
            new_keys = set(df[key_column])
            existing_keys = set(existing_df[key_column])

            keys_to_insert = new_keys - existing_keys
            keys_to_update = new_keys & existing_keys

            # Inserir novos
            df_to_insert = df[df[key_column].isin(keys_to_insert)]
            inserted = 0
            if len(df_to_insert) > 0:
                connector.write_dataframe(
                    df_to_insert,
                    table_name,
                    if_exists='append'
                )
                inserted = len(df_to_insert)

            # Atualizar existentes

```

```

        updated = 0
        if len(keys_to_update) > 0:
            # Deletar registros antigos
            keys_str = ", ".join([f"\\'{k}\\'" for k in
keys_to_update])

            connector.execute_query(
                f"DELETE FROM {table_name} WHERE {key_column} IN
({keys_str})"

            )

            # Inserir versões atualizadas
            df_to_update =
df[df[key_column].isin(keys_to_update)]
            connector.write_dataframe(
                df_to_update,
                table_name,
                if_exists='append'
            )
            updated = len(df_to_update)

            connector.connection.commit()

        result = {
            '\\inserted\\': inserted,
            '\\updated\\': updated,
            '\\total\\': inserted + updated
        }

        self.logger.log_load_complete(
            f"SQLite.{table_name} (incremental)",
            records=result[\\'total\\'],
            inserted=result[\\'inserted\\'],
            updated=result[\\'updated\\']
        )

        return result

    except Exception as e:
        self.logger.log_error(e, f"Carga incremental em {table_name}")
        raise

```

src/db_connection/sqlite_connector.py - Conector SQLite

```
"""
Conector SQLite
Autor: Diego Teodoro

Abstração para gerenciar conexões e operações no banco de dados SQLite.
"""

import sqlite3
import pandas as pd
from typing import Optional, List, Dict, Any
from pathlib import Path
from ..config import DATABASE_CONFIG, ETLLogger

class SQLiteConnector:
    """
    Gerencia conexões e operações com banco de dados SQLite.
    """

    def __init__(self, db_path: Optional[str] = None):
        """
        Inicializa o conector SQLite.

        Args:
            db_path: Caminho para o arquivo do banco de dados SQLite.
                     Se None, usa o caminho padrão da configuração.
        """
        self.db_path = db_path or DATABASE_CONFIG["sqlite"]["path"]
        self.logger = ETLLogger(__name__)
        self.connection = None

        # Criar diretório se não existir
        Path(self.db_path).parent.mkdir(parents=True, exist_ok=True)

    def connect(self) -> sqlite3.Connection:
        """
        Estabelece conexão com o banco de dados SQLite.

        Returns:
            Objeto de conexão SQLite
        """
        try:
            self.connection = sqlite3.connect(self.db_path)
            self.logger.logger.info(f"Conexão estabelecida com SQLite:
```

```

{self.db_path}")
    return self.connection
except sqlite3.Error as e:
    self.logger.log_error(e, "Conexão SQLite")
    raise

def disconnect(self):
    """Fecha a conexão com o banco de dados."""
    if self.connection:
        self.connection.close()
        self.logger.logger.info("Conexão SQLite fechada")
        self.connection = None

def execute_query(self, query: str, params: Optional[tuple] = None) ->
List[tuple]:
    """
    Executa uma query SQL e retorna os resultados.

    Args:
        query: Query SQL a ser executada
        params: Parâmetros para a query (opcional)

    Returns:
        Lista de tuplas com os resultados
    """
    if not self.connection:
        self.connect()

    try:
        cursor = self.connection.cursor()
        if params:
            cursor.execute(query, params)
        else:
            cursor.execute(query)

        results = cursor.fetchall()
        self.logger.logger.debug(f"Query executada: {query[:100]}...")
        return results
    except sqlite3.Error as e:
        self.logger.log_error(e, "Execução de query")
        raise

def read_to_dataframe(self, query: str, params: Optional[tuple] = None)
-> pd.DataFrame:
    """
    Executa uma query e retorna os resultados como DataFrame.

```

```

Args:
    query: Query SQL a ser executada
    params: Parâmetros para a query (opcional)

Returns:
    DataFrame com os resultados
"""
if not self.connection:
    self.connect()

try:
    df = pd.read_sql_query(query, self.connection, params=params)
    self.logger.logger.info(f"Dados lidos do SQLite: {len(df)}
registros")
    return df
except Exception as e:
    self.logger.log_error(e, "Leitura de dados para DataFrame")
    raise

def write_dataframe(
    self,
    df: pd.DataFrame,
    table_name: str,
    if_exists: str = 'append',
    index: bool = False
) -> int:
    """
    Escreve um DataFrame em uma tabela SQLite.

Args:
    df: DataFrame a ser escrito
    table_name: Nome da tabela de destino
    if_exists: Comportamento se a tabela existir ('fail',
'replace', 'append')
    index: Se True, escreve o índice do DataFrame

Returns:
    Número de registros escritos
"""
if not self.connection:
    self.connect()

try:
    rows_written = df.to_sql(
        table_name,

```

```

        self.connection,
        if_exists=if_exists,
        index=index
    )

    self.logger.log_load_complete(
        destination=f"SQLite.{table_name}",
        records=len(df)
    )

    return rows_written if rows_written else len(df)
except Exception as e:
    self.logger.log_error(e, f"Escrita de dados na tabela {table_name}")
    raise

def create_table(self, table_name: str, schema: Dict[str, str]):
    """
    Cria uma tabela no banco de dados.

    Args:
        table_name: Nome da tabela
        schema: Dicionário com nome das colunas e tipos de dados
            Exemplo: {'id': 'INTEGER PRIMARY KEY', 'name':
\'TEXT\'
    """
    if not self.connection:
        self.connect()

    columns = ", ".join([f"{col} {dtype}" for col, dtype in
schema.items()])
    query = f"CREATE TABLE IF NOT EXISTS {table_name} ({columns})"

    try:
        cursor = self.connection.cursor()
        cursor.execute(query)
        self.connection.commit()
        self.logger.info(f"Tabela {table_name} criada/verificada")
    except sqlite3.Error as e:
        self.logger.log_error(e, f"Criação da tabela {table_name}")
        raise

def table_exists(self, table_name: str) -> bool:
    """
    Verifica se uma tabela existe no banco de dados.

```

```

    Args:
        table_name: Nome da tabela

    Returns:
        True se a tabela existe, False caso contrário
    """
    query = """
        SELECT name FROM sqlite_master
        WHERE type='table' AND name=?
    """
    results = self.execute_query(query, (table_name,))
    return len(results) > 0

def get_table_info(self, table_name: str) -> pd.DataFrame:
    """
    Retorna informações sobre as colunas de uma tabela.

    Args:
        table_name: Nome da tabela

    Returns:
        DataFrame com informações das colunas
    """
    query = f"PRAGMA table_info({table_name})"
    return self.read_to_dataframe(query)

def create_index(self, table_name: str, column_name: str):
    """
    Cria um índice em uma coluna específica de uma tabela.
    """
    index_name = f"idx_{table_name}_{column_name}"
    query = f"CREATE INDEX IF NOT EXISTS {index_name} ON {table_name} ({column_name})"
    try:
        cursor = self.connection.cursor()
        cursor.execute(query)
        self.connection.commit()
        self.logger.logger.info(f"Índice {index_name} criado/verificado na tabela {table_name}")
    except sqlite3.Error as e:
        self.logger.log_error(e, f"Criação de índice {index_name}")
        raise

def __enter__(self):
    """Context manager - entrada."""

```



```
        self.connect()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        """Context manager - saída."""
        self.disconnect()
```

src/config/settings.py - Configurações Globais

```
import os
from pathlib import Path
from dotenv import load_dotenv

# Carregar variáveis de ambiente do arquivo .env
load_dotenv()

# Diretórios base do projeto
BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
INPUT_DIR = DATA_DIR / "input"
OUTPUT_DIR = DATA_DIR / "output"
LOGS_DIR = BASE_DIR / "logs"
SQL_DIR = BASE_DIR / "sql"

# Criar diretórios se não existirem
for directory in [INPUT_DIR, OUTPUT_DIR, LOGS_DIR]:
    directory.mkdir(parents=True, exist_ok=True)

# Configurações de banco de dados
DATABASE_CONFIG = {
    "sqlite": {
        "path": str(DATA_DIR / "output" / "ecommerce.db")
    },
    "postgres": {
        "host": os.getenv("POSTGRES_HOST", "localhost"),
        "port": os.getenv("POSTGRES_PORT", "5432"),
        "database": os.getenv("POSTGRES_DB", "etl_db"),
        "user": os.getenv("POSTGRES_USER", "postgres"),
        "password": os.getenv("POSTGRES_PASSWORD", "")
    }
}

# Configurações de API
API_CONFIG = {
    "sales_api": {
        "base_url": os.getenv("SALES_API_URL", "https://api.example.com"),
        "api_key": os.getenv("SALES_API_KEY", ""),
        "timeout": 30
    }
}

# Configurações de logging
```

```
LOG_CONFIG = {
    "level": os.getenv("LOG_LEVEL", "INFO"),
    "format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    "date_format": "%Y-%m-%d %H:%M:%S"
}

# Configurações de validação de dados
VALIDATION_CONFIG = {
    "max_null_percentage": 10.0, # Percentual máximo de nulos permitido
    "required_columns": ["id", "date", "value"],
    "date_format": "%Y-%m-%d"
}

# Configurações de retry
RETRY_CONFIG = {
    "max_attempts": 3,
    "initial_delay": 2, # segundos
    "max_delay": 10, # segundos
    "backoff_multiplier": 2
}
```

src/config/logging_config.py - Configuração de Logging

```
"""
Módulo de Logging Estruturado
Autor: Diego Teodoro

Implementação de logging personalizado para facilitar a observabilidade
e depuração do pipeline.
"""

import logging
import json
from datetime import datetime
from logging.handlers import RotatingFileHandler
from pathlib import Path
from .settings import LOGS_DIR, LOG_CONFIG


class StructuredFormatter(logging.Formatter):
    """
    Formatter customizado que gera logs em formato JSON estruturado.
    Facilita a análise e parsing de logs por ferramentas de monitoramento.
    """

    def format(self, record):
        log_entry = {
            "timestamp": datetime.fromtimestamp(record.created).isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
            "module": record.module,
            "function": record.funcName,
            "line": record.lineno
        }

        # Adicionar informações extras se existirem
        if hasattr(record, 'extra_data'):
            log_entry.update(record.extra_data)

        return json.dumps(log_entry, ensure_ascii=False)


def setup_logger(name: str, structured: bool = False) -> logging.Logger:
    """
    Configura e retorna um logger com handlers para arquivo e console.
    """
```

Args:

name: Nome do logger (geralmente __name__ do módulo)
structured: Se True, usa formato JSON estruturado

Returns:

Logger configurado

"""

```
logger = logging.getLogger(name)
logger.setLevel(getattr(logging, LOG_CONFIG["level"]))
```

```
# Evitar duplicação de handlers
```

```
if logger.handlers:
    return logger
```

```
# Formatter
```

```
if structured:
    formatter = StructuredFormatter()
else:
    formatter = logging.Formatter(
        LOG_CONFIG["format"],
        datefmt=LOG_CONFIG["date_format"]
    )
```

```
# Handler para arquivo INFO com rotação
```

```
info_handler = RotatingFileHandler(
    LOGS_DIR / "info.log",
    maxBytes=10485760, # 10MB
    backupCount=5,
    encoding='utf-8'
)
info_handler.setLevel(logging.INFO)
info_handler.setFormatter(formatter)
```

```
# Handler para arquivo ERROR
```

```
error_handler = RotatingFileHandler(
    LOGS_DIR / "error.log",
    maxBytes=10485760, # 10MB
    backupCount=5,
    encoding='utf-8'
)
error_handler.setLevel(logging.ERROR)
error_handler.setFormatter(formatter)
```

```
# Handler para console (apenas WARNING e acima)
```

```
console_handler = logging.StreamHandler()
console_handler.setLevel(logging.WARNING)
```

```
console_handler.setFormatter(formatter)
```

```
# Adicionar handlers ao logger
logger.addHandler(info_handler)
logger.addHandler(error_handler)
logger.addHandler(console_handler)
```

```
return logger
```

```
class ETLLogger:
```

```
    """
```

```
    Wrapper para logging com métodos específicos para ETL.
    Facilita o logging de eventos comuns em pipelines de dados.
    """
```

```
    def __init__(self, name: str):
        self.logger = setup_logger(name)
```

```
    def log_extraction_start(self, source: str, **kwargs):
        """
```

```
        Loga início de extração de dados.
        """
```

```
        self.logger.info(
            f"Iniciando extração de dados - Fonte: {source}",
            extra={'extra_data\': kwargs}
        )
```

```
    def log_extraction_complete(self, source: str, records: int, **kwargs):
        """
```

```
        Loga conclusão de extração de dados.
        """
```

```
        self.logger.info(
            f"Extração concluída - Fonte: {source}, Registros: {records}",
            extra={'extra_data\': {'records\': records, **kwargs}}
        )
```

```
    def log_transformation_start(self, operation: str, **kwargs):
        """
```

```
        Loga início de transformação.
        """
```

```
        self.logger.info(
            f"Iniciando transformação - Operação: {operation}",
            extra={'extra_data\': kwargs}
        )
```

```

def log_transformation_complete(self, operation: str, records_in: int,
records_out: int, **kwargs):
    """
    Loga conclusão de transformação.
    """
    self.logger.info(
        f"Transformação concluída - Operação: {operation}, "
        f"Entrada: {records_in}, Saída: {records_out}",
        extra={'extra_data\': {'records_in\': records_in,
\'records_out\': records_out, **kwargs}}
    )

def log_load_start(self, destination: str, **kwargs):
    """
    Loga início de carga de dados.
    """
    self.logger.info(
        f"Iniciando carga de dados - Destino: {destination}",
        extra={'extra_data\': kwargs}
    )

def log_load_complete(self, destination: str, records: int, **kwargs):
    """
    Loga conclusão de carga de dados.
    """
    self.logger.info(
        f"Carga concluída - Destino: {destination}, Registros:
{records}",
        extra={'extra_data\': {'records\': records, **kwargs}}
    )

def log_validation_issue(self, issue_type: str, details: str, **kwargs):
    """
    Loga problemas de validação de dados.
    """
    self.logger.warning(
        f"Problema de validação - Tipo: {issue_type}, Detalhes:
{details}",
        extra={'extra_data\': kwargs}
    )

def log_error(self, error: Exception, context: str, **kwargs):
    """
    Loga erros com contexto.
    """
    self.logger.error(

```

```
        f"Erro em {context}: {str(error)}",
        extra={'extra_data': {'error_type': type(error).__name__,
**kwargs}},
        exc_info=True
    )
```


tests/test_extract_data.py - Testes de Extração

```
import pandas as pd
import pytest
from unittest.mock import patch, MagicMock
from src.extract.extract_data import DataExtractor

@pytest.fixture
def extractor():
    return DataExtractor()

def test_extract_mock_sales_data(extractor):
    df = extractor.extract_mock_sales_data(num_records=5)
    assert isinstance(df, pd.DataFrame)
    assert len(df) == 5
    assert 'sale_id' in df.columns
    assert 'customer_id' in df.columns

def test_extract_mock_customers_data(extractor):
    df = extractor.extract_mock_customers_data(num_records=3)
    assert isinstance(df, pd.DataFrame)
    assert len(df) == 3
    assert 'customer_id' in df.columns
    assert 'first_name' in df.columns

@patch('requests.request')
def test_extract_from_api_success(mock_request, extractor):
    mock_response = MagicMock()
    mock_response.status_code = 200
    mock_response.json.return_value = {'data': [{'id': 1, 'value': 'test'}]}
    mock_request.return_value = mock_response

    result = extractor.extract_from_api('http://test.com/api')
    assert result == {'data': [{'id': 1, 'value': 'test'}]}
    mock_request.assert_called_once()

@patch('requests.request')
def test_extract_from_api_retry(mock_request, extractor):
    mock_request.side_effect = [requests.exceptions.Timeout('Timeout'),
    MagicMock(status_code=200, json=lambda: {'data': []})]

    result = extractor.extract_from_api('http://test.com/api')
    assert result == {'data': []}
    assert mock_request.call_count == 2
```

```
@patch(\'pandas.read_csv\')
def test_extract_from_csv_success(mock_read_csv, extractor):
    mock_df = pd.DataFrame({\'col1\': [1, 2], \'col2\': [\'a\', \'b\']})
    mock_read_csv.return_value = mock_df

    df = extractor.extract_from_csv(\'dummy.csv\')
    assert df.equals(mock_df)
    mock_read_csv.assert_called_once()

@patch(\'pandas.read_json\')
def test_extract_from_json_success(mock_read_json, extractor):
    mock_df = pd.DataFrame({\'col1\': [1, 2], \'col2\': [\'a\', \'b\']})
    mock_read_json.return_value = mock_df

    df = extractor.extract_from_json(\'dummy.json\')
    assert df.equals(mock_df)
    mock_read_json.assert_called_once()

@patch(\'pandas.read_parquet\')
def test_extract_from_parquet_success(mock_read_parquet, extractor):
    mock_df = pd.DataFrame({\'col1\': [1, 2], \'col2\': [\'a\', \'b\']})
    mock_read_parquet.return_value = mock_df

    df = extractor.extract_from_parquet(\'dummy.parquet\')
    assert df.equals(mock_df)
    mock_read_parquet.assert_called_once()
```

tests/test_transform_data.py - Testes de Transformação

```
import pandas as pd
import pytest
from src.transform.transform_data import DataTransformer

@pytest.fixture
def transformer():
    return DataTransformer()

def test_standardize_column_names(transformer):
    df = pd.DataFrame({"Column One": [1, 2], "Column Two": [3, 4]})
    transformed_df = transformer.standardize_column_names(df)
    assert list(transformed_df.columns) == ["column_one", "column_two"]

def test_enforce_data_types(transformer):
    df = pd.DataFrame({"col_int": [1, 2], "col_float": [1.1, 2.2]})
    type_mapping = {"col_int": "int", "col_float": "float"}
    transformed_df = transformer.enforce_data_types(df, type_mapping)
    assert transformed_df["col_int"].dtype == "int64"
    assert transformed_df["col_float"].dtype == "float64"

def test_add_calculated_columns(transformer):
    df = pd.DataFrame({"qty": [1, 2], "price": [10, 20], "discount": [0.1, 0.2]})
    calculations = {"total": "qty * price * (1 - discount)"}
    transformed_df = transformer.add_calculated_columns(df, calculations)
    pd.testing.assert_series_equal(transformed_df["total"], pd.Series([9.0, 32.0], name="total"))

def test_filter_data(transformer):
    df = pd.DataFrame({"value": [1, 5, 10, 15]})
    filtered_df = transformer.filter_data(df, "value > 5")
    assert len(filtered_df) == 2
    assert filtered_df["value"].tolist() == [10, 15]

def test_remove_duplicates(transformer):
    df = pd.DataFrame({"col1": [1, 2, 2, 3], "col2": ["a", "b", "b", "c"]})
    deduplicated_df = transformer.remove_duplicates(df)
    assert len(deduplicated_df) == 3
    assert deduplicated_df["col1"].tolist() == [1, 2, 3]

def test_handle_missing_values_drop(transformer):
    df = pd.DataFrame({"col1": [1, 2, None], "col2": ["a", None, "c"]})
    processed_df = transformer.handle_missing_values(df, strategy="drop",
```

```

columns=["col1"])
    assert len(processed_df) == 2
    assert processed_df["col1"].tolist() == [1, 2]

def test_handle_missing_values_fill(transformer):
    df = pd.DataFrame({"col1": [1, 2, None], "col2": ["a", None, "c"]})
    processed_df = transformer.handle_missing_values(df, strategy="fill",
columns=["col1"], fill_value=0)
    assert processed_df["col1"].tolist() == [1, 2, 0]

def test_join_dataframes(transformer):
    df_left = pd.DataFrame({"id": [1, 2], "value_l": [10, 20]})
    df_right = pd.DataFrame({"id": [1, 3], "value_r": [100, 300]})
    joined_df = transformer.join_dataframes(df_left, df_right, on="id",
how="left")
    assert len(joined_df) == 2
    assert "value_r" in joined_df.columns
    assert joined_df["value_r"].tolist() == [100.0, None]

def test_aggregate_data(transformer):
    df = pd.DataFrame({"category": ["A", "A", "B"], "value": [10, 20, 30]})
    aggregated_df = transformer.aggregate_data(df, group_by=["category"],
aggregations={"value": "sum"})
    assert len(aggregated_df) == 2
    assert aggregated_df["value"].tolist() == [30, 30]

```

tests/test_data_validator.py - Testes de Validação

```
import pandas as pd
import pytest
from datetime import datetime
from src.quality.data_validator import DataValidator

@pytest.fixture
def validator():
    return DataValidator()

@pytest.fixture
def sample_df():
    return pd.DataFrame({
        "id": [1, 2, 3, 4, 5],
        "name": ["A", "B", "C", "D", None],
        "value": [10, 20, 30, 40, 50],
        "date": ["2023-01-01", "2023-01-02", "2023-01-03", "2023-01-04",
"2023-01-05"],
        "amount": [100.50, 200.75, 300.00, -10.00, 500.25]
    })

def test_check_required_columns_pass(validator, sample_df):
    assert validator.check_required_columns(sample_df, ["id", "name",
"value"])

def test_check_required_columns_fail(validator, sample_df):
    assert not validator.check_required_columns(sample_df, ["id",
"non_existent_col"])

def test_check_no_nulls_in_columns_pass(validator, sample_df):
    assert validator.check_no_nulls_in_columns(sample_df, ["id", "value"])

def test_check_no_nulls_in_columns_fail(validator, sample_df):
    assert not validator.check_no_nulls_in_columns(sample_df, ["name"])

def test_check_unique_values_pass(validator, sample_df):
    assert validator.check_unique_values(sample_df, "id")

def test_check_unique_values_fail(validator):
    df = pd.DataFrame({"id": [1, 2, 2]})
    assert not validator.check_unique_values(df, "id")

def test_check_value_range_pass(validator, sample_df):
    assert validator.check_value_range(sample_df, "value", min_val=10,
```

```

max_val=50)

def test_check_value_range_fail_min(validator, sample_df):
    assert not validator.check_value_range(sample_df, "amount", min_val=0)

def test_check_value_range_fail_max(validator, sample_df):
    assert not validator.check_value_range(sample_df, "value", max_val=40)

def test_check_data_type_conformance_int_pass(validator, sample_df):
    assert validator.check_data_type_conformance(sample_df, "id", int)

def test_check_data_type_conformance_float_pass(validator, sample_df):
    assert validator.check_data_type_conformance(sample_df, "amount", float)

def test_check_data_type_conformance_str_pass(validator, sample_df):
    assert validator.check_data_type_conformance(sample_df, "name", str)

def test_check_data_type_conformance_datetime_pass(validator, sample_df):
    df_copy = sample_df.copy()
    df_copy["date"] = pd.to_datetime(df_copy["date"])
    assert validator.check_data_type_conformance(df_copy, "date", datetime)

def test_check_data_type_conformance_fail(validator, sample_df):
    assert not validator.check_data_type_conformance(sample_df, "id", str)

def test_run_all_validations_pass(validator, sample_df):
    df_copy = sample_df.copy()
    df_copy["date"] = pd.to_datetime(df_copy["date"])
    df_copy["amount"] = df_copy["amount"].apply(lambda x: max(x, 0)) # Fix
negative amount
    df_copy["name"].fillna("Unknown", inplace=True)

    config = {
        "required_columns": ["id", "name", "value", "date", "amount"],
        "no_null_columns": ["id", "value", "date", "amount"],
        "unique_columns": ["id"],
        "value_ranges": {"value": {"min": 10, "max": 50}, "amount": {"min":
0}},
        "expected_types": {"id": int, "name": str, "value": int, "date":
datetime, "amount": float}
    }
    result = validator.run_all_validations(df_copy, config)
    assert result["all_passed"]
    assert result["passed_checks"] == result["total_checks"]

def test_run_all_validations_fail(validator, sample_df):

```

```
config = {
    "required_columns": ["id", "non_existent_col"],
    "no_null_columns": ["name"],
    "unique_columns": ["id"],
    "value_ranges": {"amount": {"min": 0}},
    "expected_types": {"id": str}
}
result = validator.run_all_validations(sample_df, config)
assert not result["all_passed"]
assert result["passed_checks"] < result["total_checks"]
```

tests/test_load_data.py - Testes de Carga

```
import pandas as pd
import pytest
from unittest.mock import patch, MagicMock
from src.load.load_data import DataLoader
from src.db_connection.sqlite_connector import SQLiteConnector

@pytest.fixture
def loader():
    return DataLoader()

@pytest.fixture
def sample_df():
    return pd.DataFrame({
        "id": [1, 2, 3],
        "name": ["A", "B", "C"],
        "value": [10, 20, 30]
    })

@patch("src.db_connection.sqlite_connector.SQLiteConnector")
def test_load_to_sqlite(mock_sqlite_connector, loader, sample_df):
    mock_db_instance = MagicMock()
    mock_sqlite_connector.return_value.__enter__.return_value = mock_db_instance

    loader.load_to_sqlite(sample_df, table_name="test_table",
if_exists="replace")

    mock_db_instance.write_dataframe.assert_called_once_with(
        sample_df, "test_table", "replace", False
    )

@patch("pandas.DataFrame.to_csv")
def test_load_to_csv(mock_to_csv, loader, sample_df):
    file_path = loader.load_to_csv(sample_df, file_name="test.csv")
    mock_to_csv.assert_called_once()
    assert "test.csv" in file_path

@patch("pandas.DataFrame.to_parquet")
def test_load_to_parquet(mock_to_parquet, loader, sample_df):
    file_path = loader.load_to_parquet(sample_df, file_name="test.parquet")
    mock_to_parquet.assert_called_once()
    assert "test.parquet" in file_path
```


tests/test_sqlite_connector.py - Testes de Conexão SQLite

```
import pandas as pd
import pytest
import sqlite3
from unittest.mock import patch, MagicMock
from src.db_connection.sqlite_connector import SQLiteConnector
from src.config import DATABASE_CONFIG

@pytest.fixture
def sqlite_connector():
    # Usar um banco de dados em memória para testes
    return SQLiteConnector(db_path=":memory:")

def test_connect_and_disconnect(sqlite_connector):
    sqlite_connector.connect()
    assert sqlite_connector.connection is not None
    sqlite_connector.disconnect()
    assert sqlite_connector.connection is None

def test_execute_query(sqlite_connector):
    sqlite_connector.connect()
    sqlite_connector.execute_query("CREATE TABLE test (id INTEGER, name TEXT)")
    sqlite_connector.execute_query("INSERT INTO test (id, name) VALUES (?, ?)", (1, "Test Name"))
    result = sqlite_connector.execute_query("SELECT * FROM test")
    assert result == [(1, "Test Name")]
    sqlite_connector.disconnect()

def test_read_to_dataframe(sqlite_connector):
    sqlite_connector.connect()
    sqlite_connector.execute_query("CREATE TABLE test_df (id INTEGER, value TEXT)")
    sqlite_connector.execute_query("INSERT INTO test_df (id, value) VALUES (?, ?)", (1, "A"))
    df = sqlite_connector.read_to_dataframe("SELECT * FROM test_df")
    assert isinstance(df, pd.DataFrame)
    assert not df.empty
    assert df["id"].iloc[0] == 1
    sqlite_connector.disconnect()

def test_write_dataframe(sqlite_connector):
    sqlite_connector.connect()
    df_to_write = pd.DataFrame({"col1": [1, 2], "col2": ["a", "b"]})
```

```

        sqlite_connector.write_dataframe(df_to_write, "new_table",
if_exists="replace")
        df_read = sqlite_connector.read_to_dataframe("SELECT * FROM new_table")
        pd.testing.assert_frame_equal(df_to_write, df_read)
        sqlite_connector.disconnect()

def test_create_table(sqlite_connector):
    sqlite_connector.connect()
    schema = {"col_id": "INTEGER PRIMARY KEY", "col_name": "TEXT"}
    sqlite_connector.create_table("my_table", schema)
    assert sqlite_connector.table_exists("my_table")
    sqlite_connector.disconnect()

def test_table_exists(sqlite_connector):
    sqlite_connector.connect()
    sqlite_connector.execute_query("CREATE TABLE existing_table (id
INTEGER)")
    assert sqlite_connector.table_exists("existing_table")
    assert not sqlite_connector.table_exists("non_existing_table")
    sqlite_connector.disconnect()

def test_create_index(sqlite_connector):
    sqlite_connector.connect()
    sqlite_connector.execute_query("CREATE TABLE indexed_table (id INTEGER,
value TEXT)")
    sqlite_connector.create_index("indexed_table", "value")
    # Verificar se o índice foi criado (pode variar dependendo da versão do
SQLite)
    # Uma forma simples é tentar criar o mesmo índice novamente e verificar
se não há erro
    # ou consultar sqlite_master
    result = sqlite_connector.execute_query("SELECT name FROM sqlite_master
WHERE type='index' AND name='idx_indexed_table_value'")
    assert len(result) > 0
    sqlite_connector.disconnect()

```

requirements-core.txt - Dependências Principais

```
pandas==2.1.4
numpy==1.26.2
SQLAlchemy==2.0.23
requests==2.31.0
tenacity==8.2.3
python-dotenv==1.0.0
pyarrow==14.0.1
duckdb==0.10.0 # ou a versão mais recente compatível
```

requirements-airflow.txt - Dependências Airflow

```
apache-airflow==2.8.1
```

.env.example - Variáveis de Ambiente

```
# Exemplo de variáveis de ambiente para o pipeline ETL

# Configurações de Logging
LOG_LEVEL=INFO

# Configurações de API (exemplo)
SALES_API_URL=https://api.example.com/sales
SALES_API_KEY=your_api_key_here

# Configurações de Banco de Dados (PostgreSQL - opcional)
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=etl_db
POSTGRES_USER=postgres
POSTGRES_PASSWORD=your_password
```